

Unit Test Report

CoopCart · order-management system for Abeyrathna Farms

Framework: Jest 30 with next/jest + React Testing Library | External services (Supabase, SMTP) mocked | Date: 1 August 2026

65 passed / 0 failed / 65 total

10 test suites · all passing

1. Summary

Metric	Result
Test suites	10 passed / 10 total
Test cases	65 passed / 0 failed / 65 total
Modules covered	10 modules in src/lib
Statement coverage (tested modules)	89.0%
Function coverage (tested modules)	90.0%
Run time	~2 seconds

2. Scope — modules & functions tested

Pure-logic modules are tested directly; service modules (otp, email, auth) are tested by mocking Supabase and the mail transport so their own rules are verified without real I/O.

#	Module	Functions tested	Tests
1	pricing.ts	resolveTier, unitPriceForQty, tierRangeLabel	11
2	format.ts	formatLKR, formatDate, formatDateTime	6
3	delivery-dates.ts	computeDeliveryDates, computePickupDates, formatDateLabel	8
4	rbac.ts	rolesForAdminPath, canAccessAdminPath	7
5	labels.ts	Display label maps	5
6	env.ts	requireEnv	3
7	utils.ts	cn	4
8	otp.ts	createOtp, verifyOtp	7
9	email.ts	OTP + order emails, provider fallback	4
10	auth.ts	getSessionUser, getCurrentProfile, requireProfile, requireStaff, requireRole, getMyBusiness	10
	Total		65

3. Test cases (all 65)

pricing.ts — bulk quantity-tier pricing

- PASS** 1. resolveTier — returns null when qty is below every tier's minimum
- PASS** 2. resolveTier — picks the tier whose range contains the quantity
- PASS** 3. resolveTier — treats tier min and max as inclusive boundaries
- PASS** 4. resolveTier — uses the open-ended (no upper limit) tier for large quantities
- PASS** 5. resolveTier — ignores custom-quote tiers
- PASS** 6. resolveTier — prefers the highest matching minimum when ranges overlap
- PASS** 7. unitPriceForQty — returns the matching tier's price
- PASS** 8. unitPriceForQty — falls back to the base price when no tier matches
- PASS** 9. unitPriceForQty — returns 0 when there is neither a tier nor a base price
- PASS** 10. tierRangeLabel — shows a range label when the tier has an upper bound
- PASS** 11. tierRangeLabel — shows a "+" label for open-ended tiers

format.ts — currency & date formatting

- PASS** 12. formatLKR — formats a whole amount with the "Rs." prefix and thousands separators
- PASS** 13. formatLKR — uses the "LKR" code prefix when requested
- PASS** 14. formatLKR — keeps up to two decimal places
- PASS** 15. formatLKR — treats non-finite input (NaN) as 0
- PASS** 16. formatDate — renders a short day-month-year date
- PASS** 17. formatDateTime — includes the date, the time, and an am/pm marker

delivery-dates.ts — delivery/pickup scheduling

- PASS** 18. computeDeliveryDates — returns only the configured delivery weekdays
- PASS** 19. computeDeliveryDates — returns exactly the requested number of dates
- PASS** 20. computeDeliveryDates — offers tomorrow as the earliest date before the cut-off
- PASS** 21. computeDeliveryDates — skips to the day after tomorrow once past the cut-off
- PASS** 22. computeDeliveryDates — excludes blackout dates
- PASS** 23. computePickupDates — returns consecutive calendar days starting tomorrow
- PASS** 24. computePickupDates — excludes blackout dates
- PASS** 25. formatDateLabel — formats an ISO date as a short weekday label (e.g. "Wed 5 Aug")

rbac.ts — role-based admin access control

- PASS** 26. rolesForAdminPath — restricts the staff page to the staff-manage roles
- PASS** 27. rolesForAdminPath — applies the same restriction to nested staff paths
- PASS** 28. rolesForAdminPath — returns null (open to all staff) for unlisted paths
- PASS** 29. rolesForAdminPath — does not treat a similarly-named path as nested
- PASS** 30. canAccessAdminPath — allows admin and manager on the staff page
- PASS** 31. canAccessAdminPath — blocks other staff roles from the staff page
- PASS** 32. canAccessAdminPath — allows any role on an unrestricted path

labels.ts — display label maps

- PASS** 33. maps business types to human labels
- PASS** 34. maps size grades to human labels
- PASS** 35. maps order statuses to human labels
- PASS** 36. maps payment statuses to human labels
- PASS** 37. never produces an empty label

env.ts — environment variable access

- PASS** 38. requireEnv — returns the value when the variable is set
- PASS** 39. requireEnv — throws a clear, actionable error when the variable is missing
- PASS** 40. requireEnv — treats an empty string as missing

utils.ts — class-name helper

- PASS** 41. cn — joins multiple class names with a space
- PASS** 42. cn — drops falsy (conditional) values
- PASS** 43. cn — resolves conflicting Tailwind utilities, keeping the last
- PASS** 44. cn — supports array and object syntax

otp.ts — email one-time-passcode (Supabase mocked)

- PASS** 45. createOtp — returns a six-digit numeric code
- PASS** 46. createOtp — stores the code hashed (never plaintext) under the lower-cased email
- PASS** 47. verifyOtp — rejects when no code was stored for the email
- PASS** 48. verifyOtp — rejects and deletes an expired code
- PASS** 49. verifyOtp — rejects and deletes after too many attempts
- PASS** 50. verifyOtp — rejects a wrong code and increments the attempt counter
- PASS** 51. verifyOtp — accepts the correct code and consumes (deletes) it

email.ts — transactional email (SMTP transport mocked)

- PASS** 52. puts the OTP code in both the subject and the body
- PASS** 53. builds an order confirmation with the order number and formatted total
- PASS** 54. reports failure gracefully when the transport throws
- PASS** 55. does not deliver when no email provider is configured

auth.ts — session & role guards (Supabase + redirect mocked)

- PASS** 56. getSessionUser — returns the authenticated user
- PASS** 57. getCurrentProfile — returns null when nobody is signed in
- PASS** 58. getCurrentProfile — returns the profile row for the signed-in user
- PASS** 59. requireProfile — redirects anonymous visitors to /login
- PASS** 60. requireStaff — redirects anonymous visitors to the admin login
- PASS** 61. requireStaff — bounces customers to the customer app
- PASS** 62. requireStaff — lets a staff member through
- PASS** 63. requireRole — allows a user whose role is in the allow-list
- PASS** 64. requireRole — redirects a staff member who lacks the required role to /admin
- PASS** 65. getMyBusiness — returns null when signed out

4. Coverage (tested modules)

File	% Stmts	% Branch	% Funcs	% Lines
All files	89.03	78.63	90.00	89.89
auth.ts	92.10	68.75	100.00	90.62
delivery-dates.ts	100.00	70.83	100.00	100.00
email.ts	67.27	68.96	71.42	67.30
env.ts	76.92	75.00	100.00	100.00
format.ts	100.00	87.50	100.00	100.00
labels.ts	100.00	100.00	100.00	100.00
otp.ts	100.00	100.00	100.00	100.00
pricing.ts	100.00	94.44	100.00	100.00
rbac.ts	90.90	100.00	100.00	100.00
utils.ts	100.00	100.00	100.00	100.00

email.ts is lower because only two of its six templates and one of its two delivery providers (SMTP) are exercised; the rest share the same tested helpers. All other modules are at or near full coverage.

5. Out of scope for unit testing

These modules are thin wrappers over external systems with no branching logic of their own. Verifying them meaningfully needs integration / end-to-end tests, not unit tests — mocking them would only re-assert the implementation.

Module(s)	Reason
src/lib/data/**	Direct Supabase read queries (no logic to isolate)
src/lib/actions/**	Server actions: form handling + DB writes + revalidate/redirect
src/lib/supabase/**	Supabase client construction
src/lib/confetti.ts	Browser canvas / animation side effect

Recommended next step: add end-to-end tests (e.g. Playwright) for the key journeys — register → verify OTP → place order → admin confirms — which exercise the modules above through the real application.

6. How to reproduce

npm test	run all 65 unit tests
npm run test:coverage	run tests and print the coverage table

Test files live in src/lib/__tests__/. The runner is configured in jest.config.mjs.